

SECURITY
ASSESSMENT



OAK NETWORK

**SECURITY
ASSESSMENT
REPORT**

December 10, 2025
Neplox, <https://neplox.security>

CONTENTS

ABOUT NEPLOX	3
DISCLAIMER.....	4
1. PROJECT OVERVIEW.....	6
1.1 About Oak Network	6
1.2 Engaged Contacts.....	6
1.3 Project timeline	7
1.4 Project Targets.....	7
2. EXECUTIVE SUMMARY	8
2.1 Exposure status	8
3. SUMMARY OF FINDINGS	10
4. DETAILED RESULTS	13
4.1 Incompatible function signatures in <code>PaymentTreasuryAdapter</code>	13
4.2 Blocking <code>createPayment</code> and other payments via front-running <code>processCryptoPayment</code>	16
4.3 Missing Access Control on <code>withdraw</code> allows blocking refund mechanism	18
4.4 Campaign owner can block protocol/platform fee & reward collection in <code>PaymentTreasury</code> and <code>TimeConstrainedPaymentTreasury</code>	21
4.5 Missing treasury state validation in <code>TimeConstrainedPaymentTreasury</code>	23
4.6 Treasuries missing EIP-2771 Meta-Transaction Support.....	25
4.7 JSON Injection in NFT Metadata.....	27
4.8 Missing launch buffer validation in launch time update.....	29
4.9 Missing minimum duration validation in deadline update.....	31
4.10 Using <code>encodeWithSignature</code> Instead of <code>encodeCall</code>	33
4.11 Event order confusion after in NFT Minting.....	35
4.12 Contract reference fetched inside loop.....	37
4.13 Precision loss in reward price denormalization.....	38

ABOUT NEPLOX

Formed by like-minded, top-tier security researchers from diverse backgrounds, the Neplox Team is fueled by curiosity to explore and secure modern systems.

From international CTF winners to hardened reverse engineers and bug bounty hunters, our unique skillsets come together to offer a fresh perspective on the security of Web3 ecosystems.

To keep up to date with our latest news, announcements and research articles, please follow [@neploxaudit](https://twitter.com/neploxaudit) on x.com/neploxaudit or t.me/neploxaudit and explore our public repositories at github.com/neploxaudit.

The Neplox Team,
<https://neplox.security>

DISCLAIMER

This assessment was conducted by the Neplox Team according to the agreed-upon statement of work and aimed to cover a specific set of project goals. This project was limited in time and depended on the information and data provided by the Client. As such, this assessment does not guarantee the absence of other issues within the project scope. No warranties are made regarding the longevity of this assessment: the Neplox Team cannot make statements about code and functionality added or changed after the reviewed version of the targets. Furthermore, the Neplox Team also makes no comprehensive statements regarding the security of any functionality outside the scope and defined project goals; any discovered issues treading the boundary of the scope have been reported purely in good faith.

The Neplox Team has provided mitigation recommendations for all findings. All code samples and statements from these recommendations are intended to give ideas on how the found issues may be resolved, but should not be considered exhaustive. The Neplox Team is glad to provide additional guidance when needed, but cannot provide any guarantees regarding any fixes implemented during or after the assessment.

This file is a public version of the report on the results of the security assessment, which is meant to be publicly disclosed following a process agreed upon by the Neplox Team, Immunefi and Oak Network. This security assessment report is intended for discussion purposes only. No information contained within is to be used as legal or financial advice. No part of the report is to be interpreted as the Neplox Team's opinion and endorsement of the Client, the projects in scope, or any other referenced projects and technologies.

ENGAGEMENT OVERVIEW

1. PROJECT OVERVIEW

The Neplox Team was contracted by **Immunefi** to conduct a security assessment for **Oak Network** (hereinafter also referred to as "Client"). Following the active engagement phase, an interim report was presented and discussed to acknowledge the issues, after which this final report was delivered.

The audit was conducted in collaboration with Immunefi: <https://immunefi.com>

1.1 ABOUT OAK NETWORK

Description contributed by Oak Network, <https://www.oaknetwork.org>:

Oak Network is a decentralized, censorship-resistant crowdfunding protocol built on Celo. We provide infrastructure that any platform can integrate to enable secure, transparent, and accessible fundraising for creators, entrepreneurs, and communities.

1.2 ENGAGED CONTACTS

The following members of the Neplox Team were granted access to the project targets and engaged in the security assessment of Oak Network:

Artem Mikheev

renbou@neplox.security

Vsevolod Kokorin

slonser@neplox.security

1.3 PROJECT TIMELINE

Table 1. Project timeline

12/11/2025	Start of active engagement
20/11/2025	End of active engagement
21/11/2025	Start of fix period
04/12/2025	End of fix period

1.4 PROJECT TARGETS

The engagement involved the following targets:

- Oak Network Source Code:

Type	Code
Repositories	• https://github.com/oak-network/contracts/tree/2.0-rc • https://github.com/oak-network/safe-multisig-helper-contract
Versions	• 520b31fc063f6ba0614f3ce927544989c9136197 • 6c8990372cd72ec3c0c443afed3ce9b1ee81ea5d

2. EXECUTIVE SUMMARY

During the active engagement period of this security assessment, the Neplox Team identified **13 issues** in Oak Network, which have subsequently been reported along with free-form security-oriented discussions. As of the time this report was compiled, 12/10/2025, **all** issues and discussions have been **acknowledged** by the Client, with **all 13 fixes** reviewed by the Neplox Team.

2.1 EXPOSURE STATUS

Table 2. Summary by severity

	CRITICAL	HIGH	MEDIUM	LOW	INFO
# ISSUES	1	3	2	4	3
# FIXES	1	3	2	4	3

DETAILED RESULTS

3. SUMMARY OF FINDINGS

This table lists all the findings of the assessment and includes details on the severity and fix status of each.

Table 3. List of issues

VULNERABILITY		STATUS	SEVERITY
# 4.1	Incompatible function signatures in <code>PaymentTreasuryAdapter</code>	Fixed	Critical
# 4.2	Blocking <code>createPayment</code> and other payments via front-running <code>processCryptoPayment</code>	Fixed	High
# 4.3	Missing Access Control on <code>withdraw</code> allows blocking refund mechanism	Fixed	High
# 4.4	Campaign owner can block protocol/platform fee & reward collection in <code>PaymentTreasury</code> and <code>TimeConstrainedPaymentTreasury</code>	Fixed	High
# 4.5	Missing treasury state validation in <code>TimeConstrainedPaymentTreasury</code>	Fixed	Medium
# 4.6	Treasuries missing EIP-2771 Meta-Transaction Support	Fixed	Medium
# 4.7	JSON Injection in NFT Metadata	Fixed	Low
# 4.8	Missing launch buffer validation in launch time update	Fixed	Low
# 4.9	Missing minimum duration validation in deadline update	Fixed	Low
# 4.10	Using <code>encodeWithSignature</code> instead of <code>encodeCall</code>	Fixed	Low

# 4.11	Event order confusion after in NFT Minting	Fixed	Informational
# 4.12	Contract reference fetched inside loop	Fixed	Informational
# 4.13	Precision loss in reward price denormalization	Fixed	Informational

DETAILED RESULTS

4. DETAILED RESULTS

4.1 INCOMPATIBLE FUNCTION SIGNATURES IN PAYMENTTREASURYADAPTER

Severity	Critical
Status	Fixed in 01fcd39fe4cb8b863324910ba35bb7c28d00a1fc

Description

The `PaymentTreasuryAdapter` uses outdated function signatures that are incompatible with the current `BasePaymentTreasury` implementation, causing all adapter calls to fail.

The `createPayment()` function in the adapter encodes only 5 parameters:

```
bytes memory data = abi.encodeWithSignature(
    "createPayment(bytes32,bytes32,bytes32,uint256,uint256)",
    paymentId,
    buyerId,
    itemId,
    amount,
    expiration
);
```

However, the actual treasury function requires 8 parameters including `paymentToken`, `lineItems[]`, and `externalFees[]`:

```
function createPayment(
    bytes32 paymentId,
    bytes32 buyerId,
    bytes32 itemId,
    address paymentToken, // Missing in adapter
    uint256 amount,
    uint256 expiration,
    ICampaignPaymentTreasury.LineItem[] calldata lineItems,
    // Missing in adapter
    ICampaignPaymentTreasury.ExternalFees[] calldata externalFees
    // Missing in adapter
) public override virtual onlyPlatformAdmin(PLATFORM_HASH)
```

Similarly, `confirmPayment()` is missing the `buyerAddress` parameter, and `confirmPaymentBatch()` is missing the `buyerAddresses[]` array. When these adapter functions are called, the treasury contract receives incorrectly encoded calldata, causing function selector mismatch and immediate reversion with `CallFailed()`.

Recommendations

Update function signatures in [PaymentTreasuryAdapter.sol](#):

1. `createPayment()` - Add missing parameters:

```
// Add after itemId parameter:
address paymentToken,

// Add after expiration parameter:
IPaymentTreasury.LineItem[] calldata lineItems,
IPaymentTreasury.ExternalFees[] calldata externalFees

// Update abi.encodeWithSignature (line 17-23):
bytes memory data = abi.encodeWithSignature(
    "createPayment(bytes32,bytes32,bytes32,address,uint256,uint256,(bytes
32,uint256)[],(address,uint256)[])",
    paymentId, buyerId, itemId, paymentToken, amount, expiration,
    lineItems, externalFees
);
```

2. `confirmPayment()` - Add `buyerAddress` parameter:

```
// Add after paymentId parameter:
address buyerAddress

// Update abi.encodeWithSignature (line 53-55):
bytes memory data = abi.encodeWithSignature(
    "confirmPayment(bytes32,address)",
    paymentId, buyerAddress
);
```

3. `confirmPaymentBatch()` - Add `buyerAddresses` array:

```
// Add after paymentIds parameter:
address[] calldata buyerAddresses
```

```
// Update abi.encodeWithSignature (line 69-71):  
bytes memory data = abi.encodeWithSignature(  
    "confirmPaymentBatch(bytes32[],address[])",  
    paymentIds, buyerAddresses  
);
```

4.2 BLOCKING CREATEPAYMENT AND OTHER PAYMENTS VIA FRONT-RUNNING PROCESSCRYPTOPAYMENT

Severity	High
Status	Fixed in 45d049c75d58b31a8b568d7c5eb44d5b773bcd1f and d3f37b0b045d37dd71cafe3e5f0003707a9e145d

Description

All payment-related functions (important ones are `createPayment`, `createPaymentBatch`, `processCryptoPayment`) accept a user-controlled `paymentId` parameter that is validated to be unique. For external payments, this value comes from payment providers such as Stripe. The onchain `processCryptoPayment`, however, also uses the same `paymentId` system, which allows attackers to front-run legitimate on- and off-chain transactions and block them by using the same payment ID.

```
// All these functions check payment ID uniqueness:
function createPayment(bytes32 paymentId, ...) { ... }
function createPaymentBatch(bytes32 calldata []paymentIds, ...) { ... }
function processCryptoPayment(bytes32 paymentId, ...) { ... }

// Common check in all functions:
if(s_payment[paymentId].buyerId != ZERO_BYTES ||
s_payment[paymentId].buyerAddress != address(0)){
    revert PaymentTreasuryPaymentAlreadyExist(paymentId);
}
```

The `paymentId` is not scoped to `msg.sender`. An attacker can monitor the mempool for any pending payment transaction, extract the `paymentId`, and front-run it:

- Front-run `createPayment` with `processCryptoPayment(sameId, ..., 1 wei, ...)`
- Front-run `processCryptoPayment` with another `processCryptoPayment(sameId, ..., 1 wei, ...)`

When the legitimate transaction executes, it reverts with `PaymentTreasuryPaymentAlreadyExist`, effectively blocking all real payments to any campaign.

This vulnerability is particularly severe for Stripe integration. An attacker can monitor the platform admin's `createPayment(stripePaymentId, ...)` transactions in the mempool, extract Stripe payment IDs, and front-run them with on-chain `processCryptoPayment(stripePaymentId, ..., 1 wei, ...)`

The payment ID becomes occupied by attacker's 1 wei transaction if the front-run attack is successful. When an off-chain payment's migration to the chain via `createPayment` transaction arrives, it will fail because the ID is already taken. This can severely damage the Stripe integration - legitimate off-chain payments cannot be recorded on-chain, breaking other off-chain systems relying on this and causing potential user fund loss, or at the very least resulting in the need to process refunds for all these failed payment creations.

This is a recurrence of the same vulnerability pattern that previously existed in [KeepWhatsRaised](#) with `pledgeId` parameters.

Recommendations

Scope the payment ID to the caller by hashing it with `_msgSender()` in `processCryptoPayment` and other on-chain payment processing functions, and with the zero address for off-chain payment methods (which will allow the platform admin address to change in between these calls without breaking existing payment treasuries), similar to the same fix applied to `KeepWhatsRaised` in [commit 1d6ad87](#):

```
function createPayment(bytes32 paymentId, ...) {
    bytes32 internalPaymentId = keccak256(abi.encodePacked(paymentId,
ZERO_ADDRESS));
    if(s_payment[internalPaymentId].buyerId != ZERO_BYTES){
        revert PaymentTreasuryPaymentAlreadyExist(internalPaymentId);
    }
    s_payment[internalPaymentId] = PaymentInfo({...});
    ...
}

function processCryptoPayment(bytes32 paymentId, ...) {
    bytes32 internalPaymentId = keccak256(abi.encodePacked(paymentId,
_msgSender()));
    ...
}
```

This preserves the external `paymentId` for off-chain tracking with global uniqueness while ensuring uniqueness per caller on-chain.

4.3 MISSING ACCESS CONTROL ON WITHDRAW ALLOWS BLOCKING REFUND MECHANISM

Severity	High
Status	Fixed in 6a0f9d89c5675b783bcbb848a9940227f8f17fd6

Description

The `withdraw()` function has no access control and can be called by anyone. Combined with `PaymentTreasury._checkSuccessCondition()` always returning `true`, this allows any attacker to force withdrawals immediately after payments are received, breaking the entire refund mechanism.

```
function withdraw()
    public // No access control - anyone can call
    virtual
    override
    whenCampaignNotPaused
    whenCampaignNotCancelled
{
    if (!_checkSuccessCondition()) { // Always returns true in
PaymentTreasury
        revert PaymentTreasurySuccessConditionNotFulfilled();
    }

    address recipient = INFO.owner();
    // ...
    for (uint256 i = 0; i < acceptedTokens.length; i++) {
        address token = acceptedTokens[i];
        uint256 balance = s_availableConfirmedPerToken[token];
        if (balance > 0) {
            // Transfer funds to owner
            s_availableConfirmedPerToken[token] = 0; // Reset to zero
            IERC20(token).safeTransfer(recipient, withdrawalAmount);
        }
    }
}
```

When `withdraw()` is called, `s_availableConfirmedPerToken[token]` is reset to zero. However, `claimRefund()` checks this value to determine if refunds are possible:

```
function claimRefund(bytes32 paymentId, address refundAddress) public {
    uint256 availablePaymentAmount =
s_availableConfirmedPerToken[paymentToken];

    if (amountToRefund == 0 || availablePaymentAmount < amountToRefund) {
        revert PaymentTreasuryPaymentNotClaimable(paymentId);
        // Will always revert after withdraw
    }
    // ...
}
```

Attack scenario:

1. User makes any payment (via `processCryptoPayment()` or `createPayment()` + `confirmPayment()`)
2. Attacker immediately calls `withdraw()`
3. Campaign owner receives the funds, `s_availableConfirmedPerToken[token]` becomes 0
4. User later tries to claim refund
5. Refund fails with `PaymentTreasuryPaymentNotClaimable` because available balance is 0

This completely breaks the refund mechanism for all payment types. Any attacker can force premature withdrawals after every payment, preventing all future refunds regardless of the campaign owner's intent. The campaign owner has no way to prevent this attack or return funds collected through non-crypto payments (via `createPayment` & `confirmPayment`) except by converting crypto to fiat and manually transferring back to the buyer's bank account.

This creates legal liability issues for the platform if it provides any refund guarantees to users, as the smart contract-based refund mechanism can be completely disabled by any third party.

Recommendations

Restrict `withdraw()` to be callable only by the campaign owner, or alternatively allow both the campaign owner and platform admin (following the same pattern as [KeepWhatsRaised](#)).

Add the `onlyPlatformAdminOrCampaignOwner` modifier (similar to [KeepWhatsRaised](#)):

```
// Add this modifier to BasePaymentTreasury
modifier onlyPlatformAdminOrCampaignOwner() {
    if (
        _msgSender() != INFO.getPlatformAdminAddress(PLATFORM_HASH) &&
        _msgSender() != INFO.owner()
    ) {
        revert AccessCheckerUnauthorized();
    }
    -;
}

function withdraw()
    public
    virtual
    override
    onlyPlatformAdminOrCampaignOwner // Apply this modifier
    whenCampaignNotPaused
    whenCampaignNotCancelled
{
    if (!_checkSuccessCondition()) {
        revert PaymentTreasurySuccessConditionNotFulfilled();
    }
    // ...
}
```

This prevents arbitrary third parties from forcing premature withdrawals while allowing authorized parties (owner or platform admin) to control fund withdrawals.

4.4 CAMPAIGN OWNER CAN BLOCK PROTOCOL/PLATFORM FEE & REWARD COLLECTION IN PAYMENTTREASURY AND TIMECONSTRAINEDPAYMENTTREASURY

Severity	High
Status	Fixed in 18f697b95ac9bad47cb0f50138763167d7ecb797

Description

Campaign owners, who we can consider to be low-privileged users in Oak Network, are able to cancel their campaign through `CampaignInfo._cancelCampaign` at any point in time, including after launch and before the deadline. Since `BasePaymentTreasury` currently enforces the `whenCampaignNotPaused` and `whenCampaignNotCancelled` modifiers for all operations, including fee & reward claim methods intended for the platform/protocol (`disburseFees`, `claimNonGoalLineItems`, `claimExpiredFunds`), they can effectively block the platform they're using to host their campaign, and Oak Network itself, from being compensated according to the configured fee structure.

Besides canceling the `CampaignInfo`, owners are also able to cancel the treasuries themselves in the cases of `PaymentTreasury` and `TimeConstrainedPaymentTreasury`, both of which override `cancelTreasury` like this:

```
function cancelTreasury(bytes32 message) public override {
    if (
        _msgSender() != INFO.getPlatformAdminAddress(PLATFORM_HASH) &&
        _msgSender() != INFO.owner()
    ) {
        revert PaymentTreasuryUnauthorized();
    }
    _cancel(message);
}
```

Since `PaymentTreasury` enforces the treasury-bound `whenNotPaused` and `whenNotCancelled` modifiers for the `claimExpiredFunds` and `disburseFees` methods, they are affected through the treasury cancelations additionally to `CampaignInfo` cancelations.

This issue is especially serious due to the fact that `withdraw` calls for the payment treasuries are not time-locked and don't require explicit approval, contrary to withdrawals in the `KeepWhatsRaised` treasury (`KeepWhatsRaised.withdraw` requires the platform admin to call `approveWithdrawal`). A malicious campaign owner can wait for enough rewards to accumulate close to the campaign deadline, withdraw them, and cancel the treasury or campaign to lock the remaining protocol and platform fees and rewards.

Recommendations

It should not be possible to disable protocol and platform fee claiming completely, freezing the funds on the treasury contract forever. The `KeepWhatsRaised` treasury, for example, explicitly allows fund claims despite any present cancelations, blocking only when the treasury or campaign is "paused". Example from `KeepWhatsRaised.claimFund`:

```
function claimFund()
    external
    onlyPlatformAdmin(PLATFORM_HASH)
    whenCampaignNotPaused
    whenNotPaused
{
    bool isCancelled = s_cancellationTime > 0;
    uint256 cancellLimit = s_cancellationTime + s_config.refundDelay;
    uint256 deadlineLimit = getDeadline() + s_config.withdrawalDelay;

    if ((isCancelled && block.timestamp <= cancellLimit) || (!isCancelled
&& block.timestamp <= deadlineLimit)) {
        revert KeepWhatsRaisedNotClaimableAdmin();
    }
    ...
}
```

4.5 MISSING TREASURY STATE VALIDATION IN TIMECONSTRAINEDPAYMENTTREASURY

Severity	Medium
Status	Fixed in b35f343d0e92e51447b5d7cbd7a074ad9afc785f

Description

The [TimeConstrainedPaymentTreasury](#) contract uses wrong modifiers on its overridden functions, causing the treasury's own pause and cancel state to never be checked. All functions use `whenCampaignNotPaused` and `whenCampaignNotCancelled` which only validate the campaign's state, not the treasury's state.

For example, in `TimeConstrainedPaymentTreasury`:

```
function createPayment(...) public override whenCampaignNotPaused
whenCampaignNotCancelled {
    _checkTimeWithinRange();
    super.createPayment(...); // Already has whenCampaignNotPaused
whenCampaignNotCancelled
}
```

The base implementation in `BasePaymentTreasury` already includes these campaign state modifiers:

```
function createPayment(...) public override virtual
onlyPlatformAdmin(PLATFORM_HASH) whenCampaignNotPaused
whenCampaignNotCancelled {
    // implementation
}
```

This means the treasury's own pause/cancel state is never validated. If the treasury itself is paused or cancelled through [PausableCancellable](#) functions, all payment operations will still execute because only the campaign state is checked, not the treasury state. The same issue affects all overridden functions: `createPaymentBatch()`, `processCryptoPayment()`, `cancelPayment()`,

`confirmPayment()`, `confirmPaymentBatch()`, `claimRefund()`, `claimExpiredFunds()`, `disburseFees()`, and `withdraw()`.

In contrast, [PaymentTreasury](#) correctly uses `whenNotPaused` and `whenNotCancelled` (without "Campaign") which check the treasury's own state from [PausableCancellable](#), allowing proper treasury-level pause/cancel functionality.

Recommendations

Replace `whenCampaignNotPaused` and `whenCampaignNotCancelled` with `whenNotPaused` and `whenNotCancelled` in all overridden functions in `TimeConstrainedPaymentTreasury.sol` to match the pattern used in `PaymentTreasury.sol`:

```
function createPayment(...) public override whenNotPaused
whenNotCancelled {
    _checkTimeWithinRange();
    super.createPayment(...);
}
```

This ensures the treasury's own pause/cancel state is properly checked before calling the base implementation, which will then check the campaign's state. Without this fix, pausing or cancelling the treasury contract directly has no effect on payment operations.

4.6 TREASURIES MISSING EIP-2771 META-TRANSACTION SUPPORT

Severity	Medium
Status	Fixed in 0807ac0f5019232337760e7379ffcf1e924c028e

Description

None of the treasury contracts ([BaseTreasury](#), [BasePaymentTreasury](#), [AllOrNothing](#), [KeepWhatsRaised](#), [PaymentTreasury](#), [TimeConstrainedPaymentTreasury](#)) implement meta-transaction sender extraction, making the entire safe-multisig-helper-contracts adapter integration non-functional.

All adapter functions in [PaymentTreasuryAdapter.sol](#) correctly append `msg.sender` to calldata following the EIP-2771 pattern:

```
bytes memory dataWithSender = abi.encodePacked(data, msg.sender);  
(bool success, ) = treasury.call(dataWithSender);
```

However, [BasePaymentTreasury](#) never extracts this appended sender. It inherits from [CampaignAccessChecker](#) which uses OpenZeppelin's `Context._msgSender()` that simply returns `msg.sender` (the adapter contract address), not the actual Safe multisig address appended in calldata.

This causes all access control checks to see the `AdapterManager` contract address instead of the Safe multisig address. When the Safe calls `AdapterManager.createPayment()`, which then calls `treasury.createPayment()`, the treasury's `onlyPlatformAdmin` modifier checks `_msgSender()` and gets `address(AdapterManager)` instead of the Safe's address. Since `address(AdapterManager) != platformAdmin`, every transaction reverts with `AccessCheckerUnauthorized()`.

The issue is confirmed by [MockTreasury.sol](#) in the adapter repository, which correctly implements sender extraction for testing. This implementation was never ported to the production treasury contracts, making the entire adapter system unusable for any function with access control modifiers (`onlyPlatformAdmin`, `onlyCampaignOwner`, `onlyProtocolAdmin`).

Recommendations

Override `_msgSender()` in both `BaseTreasury` and `BasePaymentTreasury` to extract the sender from calldata only when called by a trusted forwarder (adapter contract). Simply extracting from calldata unconditionally would allow any attacker to append a fake address and bypass access control.

Implement EIP-2771 trusted forwarder pattern in both `BaseTreasury` and `BasePaymentTreasury`:

```
// Add to BasePaymentTreasury storage
address public trustedForwarder;

// Set during initialization
function initialize(..., address _trustedForwarder) external initializer {
    // ... existing initialization
    trustedForwarder = _trustedForwarder;
}

// Override _msgSender to extract sender only from trusted forwarder
function _msgSender() internal view override returns (address sender) {
    if (msg.sender == trustedForwarder && msg.data.length >= 20) {
        assembly {
            sender := shr(96, calldataload(sub(calldatasize(), 20)))
        }
    } else {
        sender = msg.sender;
    }
}
```

This ensures that only calls from the designated adapter contract can specify the original sender, while direct calls to the treasury still use `msg.sender` for access control. Without the trusted forwarder check, any attacker could call the treasury directly with an appended admin address and bypass all access restrictions.

4.7 JSON INJECTION IN NFT METADATA

Severity	Low
Status	Fixed in 37408ece3b389acd1834a105079c50aa096ad6ad

Description

The `tokenURI()` function generates NFT metadata on-chain by concatenating user-controlled strings directly into JSON without escaping special characters. Campaign owners control the NFT name through `initialize()` and can set imageURI via `setImageURI()`.

If the NFT name or image URI contains quotes (`"`), backslashes (`\`), or control characters (`\n`, `\r`, `\t`), the resulting JSON becomes invalid or malformed. This breaks NFT metadata on marketplaces.

```
function tokenURI(uint256 tokenId) public view virtual override returns
(string memory) {
    _requireOwned(tokenId);
    PledgeData memory data = s_pledgeData[tokenId];

    string memory json = string(
        abi.encodePacked(
            '{"name":"","name":', name(), " #", tokenId.toString(), // No escaping
            '","image":"","image":', s_imageURI, // No escaping
            '","attributes":[' ,
            // ...
            "]}")
    );
    return string(abi.encodePacked("data:application/json;base64,",
        Base64.encode(bytes(json))));
}
```

Example attack: Setting NFT name to `Pledge NFT`, `"malicious":"injected` produces:

```
{"name":"Pledge NFT","malicious":"injected #1","image":"..."}
```

The JSON structure is altered, and if more complex injection is used (like unescaped newlines), the JSON becomes completely invalid causing `JSON.parse()` failures in all frontend applications and marketplaces.

Additionally, this on-chain metadata generation is extremely gas-intensive. The function performs multiple `abi.encodePacked()` calls, `toString()` conversions, `toHexString()` operations, and Base64 encoding, consuming approximately 80,000-150,000 gas per call. This is 40-75x more expensive than standard off-chain metadata approaches which cost around 1,500-2,000 gas.

Recommendations

Switch to off-chain metadata storage with a base URI pattern. Add a `baseTokenURI` storage variable and modify `tokenURI()` to return `baseTokenURI + tokenId + ".json"`. This reduces gas costs by ~98% and allows proper JSON generation on the backend where escaping can be handled correctly.

If on-chain metadata must be preserved, implement JSON string escaping that handles quotes, backslashes, and control characters before concatenation. However, this will add another 20,000-50,000 gas per call.

At minimum, add validation in `initialize()` to reject NFT names and image URIs containing quotes, backslashes, or control characters below `0x20`.

4.8 MISSING LAUNCH BUFFER VALIDATION IN LAUNCH TIME UPDATE

Severity	Low
Status	Fixed in d9fd1a172745bad12015f1d01ced47ab4578f581

Description

The `updateLaunchTime()` function allows the campaign owner to set a launch time without enforcing the `CAMPAIGN_LAUNCH_BUFFER` that is required during initial campaign creation. This allows the owner to bypass the buffer period intended to give backers and platforms adequate preparation time.

During campaign creation in `CampaignInfoFactory.createCampaign()`, the launch buffer is enforced:

```
uint256 campaignLaunchBuffer =
uint256(globalParams.getFromRegistry(
    DataRegistryKeys.CAMPAIGN_LAUNCH_BUFFER));
if (campaignData.launchTime < block.timestamp + campaignLaunchBuffer) {
    revert CampaignInfoFactoryInvalidInput();
}
```

However, `updateLaunchTime()` only checks that the new time is in the future:

```
function updateLaunchTime(uint256 launchTime) external {
    if (launchTime < block.timestamp || getDeadline() <= launchTime) {
        revert CampaignInfoInvalidInput();
    }
    s_campaignData.launchTime = launchTime;
}
```

This allows a campaign owner to update the launch time to be only 1 second in the future, circumventing the buffer requirement. This can be exploited to launch campaigns with insufficient notice, preventing backers from making informed decisions or platforms from properly preparing campaign listings.

Recommendations

Add the same buffer validation used during campaign creation:

```
function updateLaunchTime(uint256 launchTime) external {
    uint256 campaignLaunchBuffer =
uint256(_getGlobalParams().getFromRegistry(DataRegistryKeys.CAMPAIGN_LAUNCH
_BUFFER));

    if (launchTime < block.timestamp + campaignLaunchBuffer ||
getDeadline() <= launchTime) {
        revert CampaignInfoInvalidInput();
    }
    s_campaignData.launchTime = launchTime;
}
```

This ensures consistent enforcement of the launch buffer regardless of whether the time is set during creation or updated later.

4.9 MISSING MINIMUM DURATION VALIDATION IN DEADLINE UPDATE

Severity	Low
Status	Fixed in 79aab5810fa9a33f74201eb41143a50c9711653a

Description

The `updateDeadline()` function allows the campaign owner to set a deadline without enforcing the `MINIMUM_CAMPAIGN_DURATION` that is required during initial campaign creation. This allows the owner to bypass the minimum duration requirement intended to ensure campaigns run for a reasonable timeframe.

During campaign creation in `CampaignInfoFactory.createCampaign()`, the minimum duration is enforced:

```
uint256 minimumCampaignDuration =
uint256(globalParams.getFromRegistry(DataRegistryKeys.MINIMUM_CAMPAIGN_DURATION));
if (campaignData.deadline < campaignData.launchTime +
minimumCampaignDuration) {
    revert CampaignInfoFactoryInvalidInput();
}
```

However, `updateDeadline()` only checks that the deadline is after the launch time:

```
function updateDeadline(uint256 deadline) external {
    if (deadline <= getLaunchTime()) {
        revert CampaignInfoInvalidInput();
    }
    s_campaignData.deadline = deadline;
}
```

This allows a campaign owner to update the deadline to be only 1 second after the launch time, circumventing the minimum duration requirement. This can be exploited to create artificially short campaigns that pressure backers into making rushed decisions without adequate time to evaluate the campaign.

Recommendations

Add the same minimum duration validation used during campaign creation:

```
function updateDeadline(uint256 deadline) external {
    uint256 minimumCampaignDuration =
uint256(_getGlobalParams().getFromRegistry(DataRegistryKeys.MINIMUM_CAMPAIG
N_DURATION));

    if (deadline <= getLaunchTime() || deadline < getLaunchTime() +
minimumCampaignDuration) {
        revert CampaignInfoInvalidInput();
    }
    s_campaignData.deadline = deadline;
}
```

This ensures consistent enforcement of the minimum campaign duration regardless of whether the deadline is set during creation or updated later.

4.10 USING ENCODEWITHSIGNATURE INSTEAD OF ENCODECALL

Severity	Low
Status	Fixed in 01fcd39fe4cb8b863324910ba35bb7c28d00a1fc and c38f75a11d2ba6f57fff6eecd37dee03d7ea8f11

Description

All 24 adapter functions use `abi.encodeWithSignature()` with string literals instead of `abi.encodeCall()`. This provides zero type safety – typos in function names or parameter types compile successfully but fail at runtime.

Issues:

- Typos in function names not caught by compiler
- Wrong parameter types compile fine, call wrong function or fail silently
- No validation that function exists on target contract
- Changes to treasury interfaces don't trigger compile errors in adapters

Affected: All 24 functions in [PaymentTreasuryAdapter.sol](#), [KeepWhatsRaisedAdapter.sol](#), [AllOrNothingAdapter.sol](#).

Current unsafe pattern (`cancelPayment` [example](#)):

```
bytes memory data = abi.encodeWithSignature(
    "cancelPayment(bytes32)", // String literal - no validation
    paymentId
);
```

Usage of `abi.encodeWithSignature` leads to silent failures, wrong function calls, broken integrations after upgrades.

Recommendations

Replace all `abi.encodeWithSignature()` calls with `abi.encodeCall()` to get compile-time type safety. The compiler will validate function names, parameter types, and

function existence, preventing typos and signature mismatches that only fail at runtime.

Current unsafe code:

```
bytes memory data = abi.encodeWithSignature(  
    "cancelPayment(bytes32)",  
    paymentId  
);
```

Should be replaced with:

```
bytes memory data = abi.encodeCall(  
    IPaymentTreasury.cancelPayment,  
    (paymentId)  
);
```

This change needs to be applied to all 24 adapter functions: 9 functions in [PaymentTreasuryAdapter.sol](#) using `IPaymentTreasury`, 12 functions in [KeepWhatsRaisedAdapter.sol](#) using `IKeeperWhatsRaised`, and 3 functions in [AllOrNothingAdapter.sol](#) using `IAllOrNothing`.

With `abi.encodeCall()`, the compiler catches typos, validates parameter types, confirms function existence, and automatically detects when treasury interfaces change. This eliminates an entire class of bugs that currently compile successfully but fail silently at runtime.

4.11 EVENT ORDER CONFUSION AFTER IN NFT MINTING

Severity	Insight
Status	Fixed in cedf1f643aa68e2e5be88d0944e4a572299ad052

Description

The `mintNFTForPledge()` function emits the `PledgeNFTMinted` event after calling `_safeMint()`. Since `_safeMint()` triggers the `onERC721Received()` callback on the recipient, a reentrancy can occur where the NFT already exists and external calls can be made before the event is emitted.

```
function mintNFTForPledge(
    address backer,
    bytes32 reward,
    address tokenAddress,
    uint256 amount,
    uint256 shippingFee,
    uint256 tipAmount
) public virtual onlyRole(MINTER_ROLE) returns (uint256 tokenId) {
    s_tokenIdCounter.increment();
    tokenId = s_tokenIdCounter.current();

    s_pledgeData[tokenId] = PledgeData({...});

    _safeMint(backer, tokenId);
    // Calls onERC721Received - reentrancy point

    emit PledgeNFTMinted(tokenId, backer, msg.sender, reward);
    // Event after callback

    return tokenId;
}
```

This breaks the expected order for off-chain indexers that rely on events to track state changes. During the `onERC721Received()` callback, the receiver can call back into the contract (like `withdraw()` or `disburseFees()` as shown in existing reentrancy tests) and emit other events before `PledgeNFTMinted` is recorded. Indexers will see these events out of order, potentially causing incorrect state tracking.

Recommendations

Emit the `PledgeNFTMinted` event before calling `_safeMint()` to ensure correct chronological event ordering. The OpenZeppelin ERC721 `Transfer` event will still be emitted during `_safeMint()` after the custom event, which is the expected pattern.

Alternatively, add a `nonReentrant` modifier to `mintNFTForPledge()` to prevent any callbacks from executing during minting, though this adds gas costs and doesn't solve the fundamental event ordering issue.

4.12 CONTRACT REFERENCE FETCHED INSIDE LOOP

Severity	Insight
Status	Fixed in 68b7ca08a22785bbc8e7c54aa002f87a01d72d55

Description

The `updateSelectedPlatform()` function calls `_getGlobalParams()` inside a loop. Each call loads the global params contract address from storage and performs an external call.

```
for (uint256 i = 0; i < platformDataKey.length; i++) {  
    isValid = _getGlobalParams().checkIfPlatformDataKeyValid(  
        // External call in loop  
        platformDataKey[i]  
    );  
}
```

Each `_getGlobalParams()` call reads from storage and returns a contract reference. While the external call cost dominates, the repeated storage read adds unnecessary overhead. For loops with many platform data keys, this accumulates significantly.

Recommendations

Cache `_getGlobalParams()` before the loop:

```
IGlobalParams globalParams = _getGlobalParams();  
for (uint256 i = 0; i < platformDataKey.length; i++) {  
    isValid = globalParams.checkIfPlatformDataKeyValid(platformDataKey[i]);  
}
```

This eliminates repeated storage reads and saving gas.

4.13 PRECISION LOSS IN REWARD PRICE DENORMALIZATION

Severity	Isight
Status	Fixed in 13bfbef0fc803a6c6cce5de329ddf4490e3ae3ec

Description

The `_denormalizeAmount()` function rounds down when converting from 18 decimals to token decimals, causing backers to pay slightly less than the intended reward price. This occurs in `AllOrNothing.pledge()` and `KeepWhatsRaised._pledge()` when processing reward pledges.

```
function _denormalizeAmount(address token, uint256 amount) internal view
returns (uint256) {
    uint8 decimals = IERC20Metadata(token).decimals();

    if (decimals < STANDARD_DECIMALS) {
        return amount / (10 ** (STANDARD_DECIMALS - decimals));
        // Rounds down
    }
    // ...
}
```

When reward prices are stored in 18 decimals and converted to tokens with fewer decimals like USDC (6 decimals), the division truncates fractional amounts. For example:

- Reward price: 10.123456789 USDC = 10123456789000000000 (18 decimals)
- After denormalization: 10123456789000000000 / 10¹² = 10123456 (10.123456 USDC)
- Lost amount: 0.000000789 USDC

The campaign owner loses the fractional difference on every pledge. While the individual loss per transaction is minimal, the fix is straightforward and should be implemented to ensure accurate pricing.

Recommendations

Round up instead of down to ensure the campaign owner receives at least the intended amount:

```
function _denormalizeAmount(address token, uint256 amount) internal view
returns (uint256) {
    uint8 decimals = IERC20Metadata(token).decimals();

    if (decimals < STANDARD_DECIMALS) {
        uint256 divisor = 10 ** (STANDARD_DECIMALS - decimals);
        return (amount + divisor - 1) / divisor; // Round up
    }
    // ...
}
```

Alternatively, validate during reward creation that prices convert cleanly without remainders when using tokens with fewer decimals.